

Hands-On

Exercise 5: Task Management System

In this module I was able to successfully implement, execute and test a Task Management System in .NET is developed by creating a project called TaskManagementSystem. I developed a Task class with attributes like TaskId, TaskName, and Status to handle the information of the tasks effectively. I learned about various types of linked lists, such as Singly Linked List and Doubly Linked List, their structure and working. I then used Singly Linked List to store the tasks and created efficient functions to add, search, traverse and delete task records. Once implemented, I performed and tested different operations to verify the correct management of the tasks in the system. I also studied the time complexity of various operations and the pros and cons of linked list over arrays like dynamic memory allocation, efficient insertion and deletion. This hands-on activity was a great way to appreciate the significance of linked lists and the need for appropriate data structures in developing efficient and maintainable .NET applications.

Coding:

```
<?xml version='1.0' encoding='utf-8'>
<Project Sdk='Microsoft.NET.Sdk'>
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

```
PS D:\Cognizant Hands-On\MODULE 2\TaskManagementSystem> dotnet build
Restore complete (0.3s)
TaskManagementSystem net8.0 succeeded (0.1s) -> bin\Debug\net8.0\TaskManagementSystem.dll

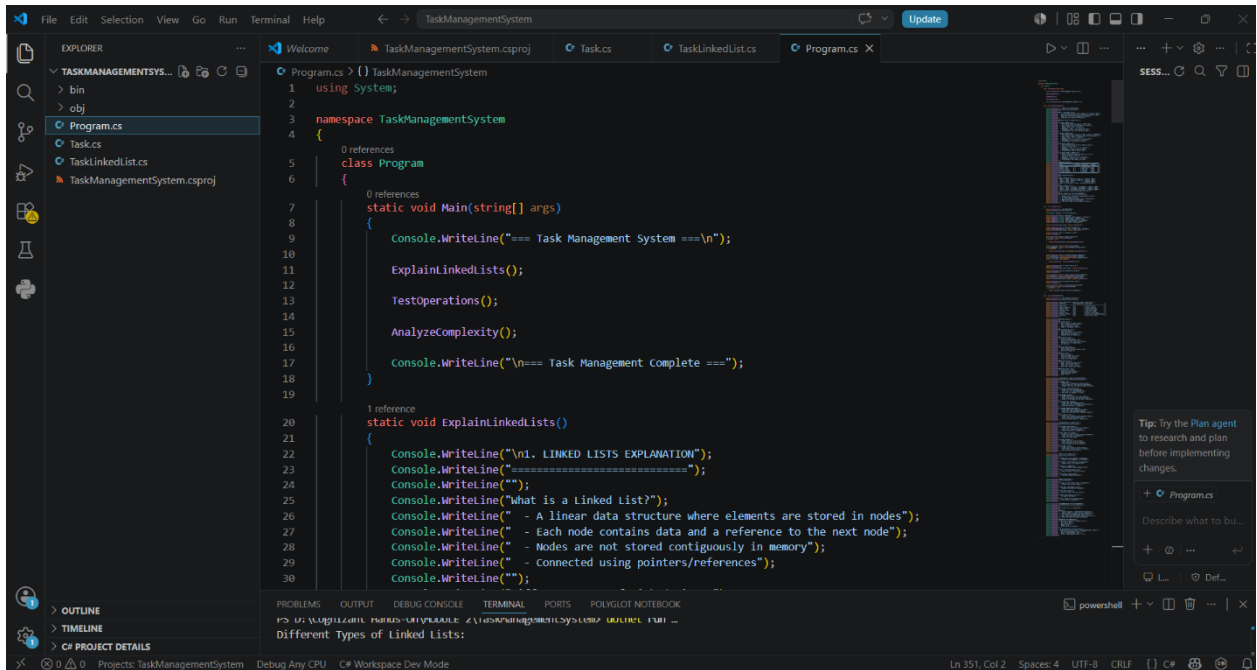
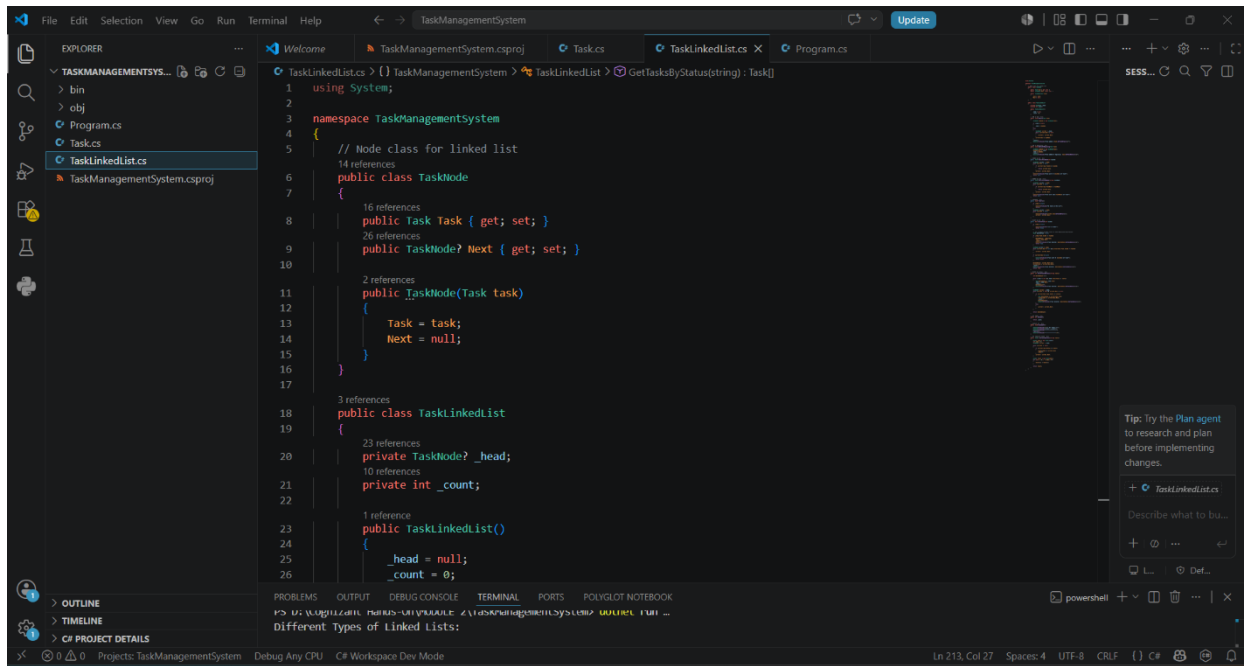
Build succeeded in 0.9s
PS D:\Cognizant Hands-On\MODULE 2\TaskManagementSystem> dotnet run
=== Task Management System ===

1. LINKED LISTS EXPLANATION
=====

What is a Linked List?
- A linear data structure where elements are stored in nodes
- Each node contains data and a reference to the next node
- Nodes are not stored contiguously in memory
- Connected using pointers/references

Different Types of Linked Lists:

1. Singly Linked List
- Each node has data and pointer to NEXT node
- Can traverse only in one direction (forward)
- Memory: Node = Data + 1 pointer
- Example: A -> B -> C -> D -> null
```



Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MODULE 2\TaskManagementSystem> dotnet build
Restore complete (0.3s)
TaskManagementSystem net8.0 succeeded (0.1s) -> bin\Debug\net8.0\TaskManagementSystem.dll

Build succeeded in 0.9s
PS D:\Cognizant Hands-On\MODULE 2\TaskManagementSystem> dotnet run
=== Task Management System ===

1. LINKED LISTS EXPLANATION
=====

What is a Linked List?
- A linear data structure where elements are stored in nodes
- Each node contains data and a reference to the next node
- Nodes are not stored contiguously in memory
- Connected using pointers/references

Different Types of Linked Lists:

1. SINGLY LINKED LIST
- Each node has data and pointer to NEXT node
- Can traverse only in one direction (forward)
- Memory: Node = Data + 1 pointer
- Example: A -> B -> C -> D -> null
- Advantages: Simple, less memory per node
- Disadvantages: Can only traverse forward

2. DOUBLY LINKED LIST
- Each node has data + pointer to NEXT + pointer to PREVIOUS
- Can traverse in both directions (forward and backward)
- Memory: Node = Data + 2 pointers
- Example: null <- A <-> B <-> C <-> D -> null
- Advantages: Can traverse both directions
- Disadvantages: More memory per node

3. CIRCULAR LINKED LIST
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
3. CIRCULAR LINKED LIST
- Last node points back to first node (no null)
- Forms a circle/cycle
- Example: A -> B -> C -> D -> A (circle)
- Advantages: Can start from any node
- Disadvantages: Need to detect cycle

4. CIRCULAR DOUBLY LINKED LIST
- Doubly linked list where last points to first
- First also points to last
- Example: A <-> B <-> C <-> D (circular)
- Advantages: Bidirectional + circular
- Disadvantages: Most memory per node

Comparison Table:


| Type            | Pointers | Traversal    | Memory |
|-----------------|----------|--------------|--------|
| Singly Linked   | 1        | Forward only | Less   |
| Doubly Linked   | 2        | Both dirs    | More   |
| Circular Linked | 1        | Circular     | Less   |
| Circular Doubly | 2        | Circular     | More   |



Memory Representation:

Singly Linked List:
Node 1: [Task1 | pointer to Node 2] -> Address: 1000
Node 2: [Task2 | pointer to Node 3] -> Address: 1500
Node 3: [Task3 | null] -> Address: 2000
Nodes are scattered in memory, connected by pointers

Doubly Linked List:
Node 1: [Task1 | prev=null | next=Node2] -> Address: 1000
Node 2: [Task2 | prev=Node1 | next=Node3] -> Address: 1500
Node 3: [Task3 | prev=Node2 | next=null] -> Address: 2000
```



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
D:\c\... D:\c\... D:\c\...
3. TIME COMPLEXITY ANALYSIS
=====

Time Complexity of Operations (Singly Linked List):
| Operation | Time Complexity | Description |
|-----|-----|-----|
| Add at End | O(n) | Traverse to end |
| Add at Beginning | O(1) | Direct insertion |
| Search by ID | O(n) | Traverse and check |
| Search by Name | O(n) | Traverse and check |
| Traverse | O(n) | Visit all n nodes |
| Delete by ID | O(n) | Search + unlink node |
| Delete by Status | O(n) | Traverse all, delete matches |
| Get Count | O(1) | Return stored count |

Detailed Analysis:

Add at End: O(n)
- Must traverse from head to end
- Visit n nodes in worst case
- Then add new node at end
- Total: n traversals = O(n)

Add at Beginning: O(1)
- Create new node
- Point new node to current head
- Update head to new node
- Constant time, no traversal

Search by ID: O(n)
- Start from head node
- Traverse and check each node
- Worst case: task at end or not found
- Average case: n/2 comparisons
- n/2 = O(n)

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
- No pointer overhead

5. Small datasets with random access
- O(1) access is important

RECOMMENDATION FOR TASK MANAGEMENT:
=====

USE: Linked List (as implemented)

REASONS:
1. Tasks are dynamic - added/deleted frequently
2. Unknown number of tasks (can grow indefinitely)
3. O(1) insertion at beginning (urgent tasks)
4. Don't need index-based access (search by ID/name)
5. Memory efficient for sparse task data

ALTERNATIVE FOR PRODUCTION:
Use List<T> (C# built-in dynamic array):
- O(1) add at end
- O(1) index access
- Dynamic size
- Better cache performance

OR use Dictionary for O(1) search:
Dictionary<int, Task> tasks = new Dictionary<int, Task>();
- Add: tasks.Add(taskId, task) - O(1)
- Search: tasks[taskId] - O(1)
- Delete: tasks.Remove(taskId) - O(1)

=== Task Management Complete ===
PS D:\Cognizant Hands-On\MODULE 2\TaskManagementSystem>

```